

DTE Platform — Commit 0 Generation Manifest

This is the definitive, implementation-grade Generation Manifest for Commit 0. All architectural decisions, boundaries, and technology selections are closed.

This document provides the exact file contents required to generate the repository skeleton. Upon execution of this manifest, the repository will be ready for the first feature branch.

1. DtePlatform.sln

Solution Folders:

- src
- tests
- Solution Items

Project Registration Order:

1. src\DTE.Domain\DTE.Domain.csproj
2. src\DTE.Application\DTE.Application.csproj
3. src\DTE.Infrastructure\DTE.Infrastructure.csproj
4. src\DTE.Api\DTE.Api.csproj
5. tests\DTE.Domain.Tests\DTE.Domain.Tests.csproj
6. tests\DTE.Application.Tests\DTE.Application.Tests.csproj
7. tests\DTE.Infrastructure.Tests\DTE.Infrastructure.Tests.csproj
8. tests\DTE.Api.Tests\DTE.Api.Tests.csproj
9. tests\DTE.Architecture.Tests\DTE.Architecture.Tests.csproj

Project Inclusion Strategy:

The solution strictly binds the projects together. Solution Items includes .editorconfig, Directory.Build.props, Directory.Packages.props, global.json, NuGet.config, and README.md.

2. Directory.Build.props

XML

<Project>

<PropertyGroup>

<TargetFramework> net6.0 </TargetFramework>

<ImplicitUsings> enable </ImplicitUsings>

<Nullable> enable </Nullable>

<TreatWarningsAsErrors> true </TreatWarningsAsErrors>

<EnforceCodeStyleInBuild> true </EnforceCodeStyleInBuild>

```

    <Deterministic>    </Deterministic>
    <AnalysisLevel>    </AnalysisLevel>
    <GenerateDocumentationFile>    </GenerateDocumentationFile>
    <NoWarn>    </NoWarn>
  </PropertyGroup>
</Project>

```

3. Directory.Packages.props

```

XML
<Project>
  <PropertyGroup>
    <ManagePackageVersionsCentrally>    </ManagePackageVersionsCentrally>
  </PropertyGroup>
  <ItemGroup>
    <!-- Application Dependencies -->
    <!-- Justification: Core CQRS implementation for state-mutating commands and queries -->
    <PackageVersion Include="MediatR" Version="12.2.0" />
    <!-- Justification: Pre-execution validation for structural command integrity -->
    <PackageVersion Include="FluentValidation.DependencyInjectionExtensions" Version="11.9.0" />

    <!-- Infrastructure Dependencies -->
    <!-- Justification: Core persistence engine -->
    <PackageVersion Include="Microsoft.EntityFrameworkCore" Version="8.0.4" />
    <!-- Justification: Core persistence engine design-time tools for migrations -->
    <PackageVersion Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.4" />
    <!-- Justification: PostgreSQL provider, required for JSONB and BYTEA support -->
    <PackageVersion Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="8.0.2" />
    <!-- Justification: Background processing queue backed by PostgreSQL -->
    <PackageVersion Include="Hangfire.PostgreSql" Version="1.20.8" />
    <PackageVersion Include="Hangfire.AspNetCore" Version="1.8.12" />

    <!-- API Dependencies -->
    <!-- Justification: Structured JSON logging to filesystem/stdout -->
    <PackageVersion Include="Serilog.AspNetCore" Version="8.0.1" />
    <!-- Justification: Stateless JWT authentication handling -->
    <PackageVersion Include="Microsoft.AspNetCore.Authentication.JwtBearer" Version="8.0.4" />

```

```

<!-- Testing Dependencies -->
<!-- Justification: xUnit is the industry standard .NET test runner -->
<PackageVersion Include="xUnit" Version="2.7.0" />
<PackageVersion Include="xunit.runner.visualstudio" Version="2.5.7" />
<PackageVersion Include="Microsoft.NET.Test.Sdk" Version="17.9.0" />
<!-- Justification: Explicit, readable assertion syntax -->
<PackageVersion Include="FluentAssertions" Version="6.12.0" />
<!-- Justification: Mocking framework for Application Layer isolation -->
<PackageVersion Include="NSubstitute" Version="5.1.0" />
<!-- Justification: Architectural boundary enforcement during CI/CD -->
<PackageVersion Include="NetArchTest.Rules" Version="1.3.2" />
<!-- Justification: Real PostgreSQL instances for physical integration tests -->
<PackageVersion Include="Testcontainers.PostgreSql" Version="3.8.0" />
<!-- Justification: In-memory test host for API E2E tests -->
<PackageVersion Include="Microsoft.AspNetCore.Mvc.Testing" Version="8.0.4" />
</ItemGroup>
</Project>

```

4. .editorconfig

```

Ini, TOML
root true

[*.*]
# Formatting
indent_style      space
indent_size       4
end_of_line       crlf
charset           utf-8
trim_trailing_whitespace true
insert_final_newline true

# Clean Architecture Enforcements
csharp_style_namespace_declarations = file_scoped_error
csharp_style_expression_bodied_methods = when_on_single_line_suggestion
csharp_style_expression_bodied_constructors = false_error
csharp_style_expression_bodied_properties = true_suggestion

```

```
# Naming Conventions
```

```
dotnet_naming_rule.interface_should_be_begins_with_i.severity = error
```

```
dotnet_naming_rule.interface_should_be_begins_with_i.symbols = interface
```

```
dotnet_naming_rule.interface_should_be_begins_with_i.style = begins_with
```

```
dotnet_naming_symbols.interface.applicable_kinds = interface
```

```
dotnet_naming_symbols.interface.applicable_accessibilities = public internal private protected
```

```
protected internal private protected
```

```
dotnet_naming_style.begins_with_i.required_prefix =
```

```
dotnet_naming_style.begins_with_i.capitalization = pascal_case
```

```
dotnet_naming_rule.private_or_internal_field_should_be_begins_with__severity = error
```

```
dotnet_naming_rule.private_or_internal_field_should_be_begins_with__.symbols =
```

```
private_or_internal_field
```

```
dotnet_naming_rule.private_or_internal_field_should_be_begins_with__.style = begins_with
```

```
dotnet_naming_symbols.private_or_internal_field.applicable_kinds = field
```

```
dotnet_naming_symbols.private_or_internal_field.applicable_accessibilities = internal private
```

```
private protected
```

```
dotnet_naming_style.begins_with__.required_prefix =
```

```
dotnet_naming_style.begins_with__.capitalization = camel_case
```

5. global.json

```
JSON
```

```
{
```

```
  "sdk"
```

```
    "version" "8.0.204"
```

```
    "rollForward" "disable"
```

```
}
```

```
|
```

6. .gitignore

Plaintext

IDEs

Idea

vscode

use

src

Build results

Build

Obj

macOS

DS_Store

Docker

dockerignore

docker-compose.override.yml

Secret

appsettings.Development.json

appsettings.Local.json

pl

perl

key

7. NuGet.config

XML

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<configuration>
```

```
<packageSources>
```

```
<clear />
```

```

    <add key="nuget.org" value="https://api.nuget.org/v3/index.json" protocolVersion="3" />
  </packageSources>
  <packageRestore>
    <add key="enabled" value="True" />
    <add key="automatic" value="True" />
  </packageRestore>
</configuration>

```

8. Dockerfile

```

Dockerfile
# Stage 1: Build
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

# Restore packages based on CPM
COPY Directory.Packages.props Directory.Build.props ./
COPY src/DTE.Domain/DTE.Domain.csproj src/DTE.Domain/
COPY src/DTE.Application/DTE.Application.csproj src/DTE.Application/
COPY src/DTE.Infrastructure/DTE.Infrastructure.csproj src/DTE.Infrastructure/
COPY src/DTE.Api/DTE.Api.csproj src/DTE.Api/
RUN dotnet restore src/DTE.Api/DTE.Api.csproj

# Copy source and build
COPY src/ src/
RUN dotnet publish src/DTE.Api/DTE.Api.csproj -c Release -o /app/publish /p:UseAppHost=false

# Stage 2: Runtime
FROM mcr.microsoft.com/dotnet/aspnetcore:8.0 AS runtime
WORKDIR /app
COPY --from=build /app/publish .

# Security hardening: Non-root execution
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser

EXPOSE 8080
ENV ASPNETCORE_URLS=http://*:*:8080

```

```
# Health Support
HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
  CMD wget --quiet --tries=1 --spider http://localhost:8080/health || exit 1
```

```
ENTRYPOINT ["dotnet", "DTE.Api.dll"]
```

9. docker-compose.dev.yml

```
YAML
version: '3.8'

services:
  postgres:
    image: postgres:15-alpine
    container_name: dte_postgres_dev
    environment:
      POSTGRES_USER: dte_user
      POSTGRES_PASSWORD: dte_dev_pass
      POSTGRES_DB: dte_platform
    ports:
      - "5432:5432"
    volumes:
      - postgres_dev_data:/var/lib/postgresql/data
      - ./scripts/db-init.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: "CMD-SHELL" "pg_isready -U dte_user -d dte_platform"
      interval: 5s
      timeout: 5s
      retries: 5

volumes:
  postgres_dev_data:
```

10. docker-compose.prod.yml

YAML

version: '3.8'

services:

api:

build:

context: .

dockerfile: deploy/docker/Dockerfile

container_name: dte_api_prod

restart: always

ports:

- "8080:8080"

environment:

- ASPNETCORE_ENVIRONMENT=Production

-

DTE_Database_ConnectionString=Host=postgres;Database=dte_platform;Username=dte_user;P

assword=\${DTE_DB_PASSWORD}

depends_on:

postgres:

condition: service_healthy

networks:

- dte_network

postgres:

image: postgres:15-alpine

container_name: dte_postgres_prod

restart: always

environment:

POSTGRES_USER: dte_user

POSTGRES_PASSWORD: \${DTE_DB_PASSWORD}

POSTGRES_DB: dte_platform

volumes:

- postgres_prod_data:/var/lib/postgresql/data

healthcheck:

test: "CMD-SHELL" "pg_isready -U dte_user -d dte_platform"

interval: 10s

timeout: 5s

retries: 5


```
networks:  
  - dte_network
```

```
volumes:  
  postgres_prod_data:
```

```
networks:  
  dte_network:  
    driver: bridge
```

11. GitHub Actions

.github/workflows/ci.yml

YAML

```
name: CI Build & Test
```

```
on:  
  push:  
    branches: "main"
```

```
jobs:  
  build:  
    runs-on: ubuntu-latest
```

```
steps:  
  - uses: actions/checkout@v3
```

```
  - name: Setup .NET  
    uses: actions/setup-dotnet@v3  
    with:  
      global-json-file: global.json
```

```
  - name: Verify Formatting (.editorconfig)  
    run: dotnet format --verify-no-changes
```

```
  - name: Restore dependencies
```

```
run: dotnet restore DtePlatform.sln
```

```
- name: Build
```

```
run: dotnet build DtePlatform.sln --no-restore -c Release
```

```
- name: Run Domain Tests
```

```
run: dotnet test tests/DTE.Domain.Tests --no-build -c Release
```

```
- name: Run Application Tests
```

```
run: dotnet test tests/DTE.Application.Tests --no-build -c Release
```

```
- name: Run Infrastructure Integration Tests
```

```
run: dotnet test tests/DTE.Infrastructure.Tests --no-build -c Release
```

.github/workflows/pr-validation.yml

YAML

```
name: PR Validation
```

```
on:
```

```
  pull_request:
```

```
    branches: "main"
```

```
jobs:
```

```
  validate:
```

```
    runs-on: ubuntu-latest
```

```
  steps:
```

```
    - uses: actions/checkout@v3
```

```
    - name: Setup .NET
```

```
      uses: actions/setup-dotnet@v3
```

```
      with:
```

```
        global-json-file: global.json
```

```
    - name: Verify Formatting (.editorconfig)
```

```
      run: dotnet format --verify-no-changes
```

- name: Restore dependencies

run: dotnet restore DtePlatform.sln

- name: Build (TreatWarningsAsErrors)

run: dotnet build DtePlatform.sln --no-restore -c Release

- name: Run Architecture Validation

run: dotnet test tests/DTE.Architecture.Tests --no-build -c Release

- name: Run Full Test Suite

run: dotnet test DtePlatform.sln --no-build -c Release

.github/workflows/architecture-validation.yml

YAML

name: Architecture Enforcement

on:

push:

branches: **!**/*!****

jobs:

enforce-boundaries:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3

- name: Setup .NET

uses: actions/setup-dotnet@v3

with:

global-json-file: global.json

- name: Build Architecture Tests

run: dotnet build tests/DTE.Architecture.Tests -c Release

```
- name: Run Layer Boundaries Validation
```

```
run: dotnet test tests/DTE.Architecture.Tests -c Release
```

12. Program.cs

File: src/DTE.Api/Program.cs

```
C#
using DTE.Application;
using DTE.Infrastructure;
using Microsoft.AspNetCore.Builder;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Serilog;

var builder = WebApplication.CreateBuilder(args);

// Configure Serilog
builder.Host.UseSerilog((context, configuration) =>
    configuration.ReadFrom(Configuration(context.Configuration)));

// Dependency Injection Bootstrap
builder.Services.AddApplication();
builder.Services.AddInfrastructure(builder.Configuration);

builder.Services.AddControllers();
builder.Services.AddHealthChecks();

var app = builder.Build();

app.UseSerilogRequestLogging();

app.UseRouting();

// Health Check Endpoint (Required for Commit 0)
app.MapHealthChecks("/health");

app.MapControllers();
```

```
app.Run();
```

```
// Required for Testcontainers/WebApplicationFactory integration  
public partial class Program
```

13. Dependency Injection Bootstrap

File: src/DTE.Application/DependencyInjection.cs

```
C#  
using FluentValidation;  
using Microsoft.Extensions.DependencyInjection;  
using System.Reflection;  
  
namespace DTE.Application  
  
public static class DependencyInjection  
{  
    public static IServiceCollection AddApplication(this IServiceCollection services)  
    {  
        var assembly = Assembly.GetExecutingAssembly();  
  
        services.AddMediatR(config =>  
        {  
            config.RegisterServicesFromAssembly(assembly);  
        });  
  
        services.AddValidatorsFromAssembly(assembly);  
  
        return services;  
    }  
}
```

File: src/DTE.Infrastructure/DependencyInjection.cs

```
C#
```

```
using DTE.Infrastructure.Persistence
```

```
using Hangfire
```

```
using Hangfire.PostgreSql
```

```
using Microsoft.EntityFrameworkCore
```

```
using Microsoft.Extensions.Configuration
```

```
using Microsoft.Extensions.DependencyInjection
```

```
namespace DTE.Infrastructure
```

```
public static class DependencyInjection
```

```
{
```

```
    public static IServiceCollection AddInfrastructure(this IServiceCollection services, IConfiguration configuration)
```

```
    {
```

```
        var connectionString = configuration.GetConnectionString("Database")
```

```
        ?? throw new InvalidOperationException("Database connection string is missing.")
```

```
        services.AddDbContext<ApplicationDbContext>(options =>
```

```
            options.UseNpgsql(connectionString))
```

```
        services.AddHangfire(config => config
```

```
            .SetDataCompatibilityLevel(CompatibilityLevel.Version_180)
```

```
            .UseSimpleAssemblyNameTypeSerializer()
```

```
            .UseRecommendedSerializerSettings()
```

```
            .UsePostgreSqlStorage(options =>
```

```
                options
```

```
                    .UseNpgsqlConnection(connectionString))
```

```
            ))
```

```
        services.AddHangfireServer()
```

```
        return services
```

```
    }
```

```
}
```

14. ApplicationDbContext Stub

File: src/DTE.Infrastructure/Persistence/ApplicationDbContext.cs

```
C#
using Microsoft.EntityFrameworkCore;

namespace DTE.Infrastructure.Persistence

public sealed class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
    }

    // Configuration bindings will be registered here in Commit 2.

    modelBuilder.ApplyConfigurationsFromAssembly<typeof(ApplicationDbContext) Assembly>();
}
```

15. Architecture Test Bootstrap

File: tests/DTE.Architecture.Tests/LayerTests.cs

```
C#
using FluentAssertions;
using NetArchTest.Rules;
using Xunit;
```

```
namespace DTE.Architecture.Tests
```

```
public class LayerTests
```

```
private const string DomainNamespace = "DTE.Domain"
```

```
private const string ApplicationNamespace = "DTE.Application"
```

```
private const string InfrastructureNamespace = "DTE.Infrastructure"
```

```
private const string ApiNamespace = "DTE.Api"
```

```
Fact
```

```
public void Domain_Should_Not_Have_Dependencies_On_Other_Layers()
```

```
var result = Types.InCurrentDomain()
```

```
.That()
```

```
.ResideInNamespace(DomainNamespace)
```

```
.ShouldNot()
```

```
.HaveDependencyOnAny(ApplicationNamespace, InfrastructureNamespace,
```

```
ApiNamespace)
```

```
.GetResult()
```

```
result.IsSuccessful.Should().BeTrue()
```

```
Fact
```

```
public void Application_Should_Not_Have_Dependencies_On_Infrastructure_Or_Api()
```

```
var result = Types.InCurrentDomain()
```

```
.That()
```

```
.ResideInNamespace(ApplicationNamespace)
```

```
.ShouldNot()
```

```
.HaveDependencyOnAny(InfrastructureNamespace, ApiNamespace)
```

```
.GetResult()
```

```
result.IsSuccessful.Should().BeTrue()
```

```
Fact
```

```
public void Infrastructure_Should_Not_Have_Dependency_On_Api()
```

```
var result = Types.InCurrentDomain()
```

```
.That()
```

```
.ResideInNamespace(InfrastructureNamespace)
```



```

        .ShouldNot()
        .HaveDependencyOn(ApiNamespace)
        .GetResult()

        result.IsSuccessful.Should().BeTrue();
    }
}

```

16. README.md

```

Markdown
# DTE Platform (V1)

The single, legally compliant issuance engine for Factura Electronica (01) and
Comprobante de Crédito Fiscal (03).

## Prerequisites
- .NET 8 SDK
- Docker Desktop

## Local Development Bootstrap

1. Clone the repository
2. Boot the database
```bash
docker-compose -f deploy/docker-compose.dev.yml up -d

```

3. Restore dependencies:

```

Bash
dotnet restore DtePlatform.sln

```

4. Run the API:

## Bash

```
dotnet run -p:obj\src\DTE.Api
```

### 5. Verify health:

Navigate to <http://localhost:5001/health>

## Automated Tests

- Unit Tests: `dotnet test tests/DTE.Domain.Tests`
- Architecture Verification: `dotnet test tests/DTE.Architecture.Tests`
- Full Suite (Requires Docker): `dotnet test DtePlatform.sln`



## # 17. Commit 0 Verification Checklist

Before accepting Commit 0, the Lead Engineer must verify:

- [ ] All files specified in this manifest exist in the exact directory structure
- [ ] `dotnet build DtePlatform.sln` completes with 0 errors and 0 warnings
- [ ] `dotnet test tests/DTE.Architecture.Tests` passes successfully
- [ ] `docker-compose -f deploy/docker-compose.dev.yml up -d` successfully starts PostgreSQL on port 5432
- [ ] The `.github/workflows` YAML files are valid
- [ ] `global.json` locks the repository to .NET 8.0.x

Status: Ready for Repository Generation